大学生创新创业训练项目 · 开发报告

长株潭「标点地图」文旅小程序

地图 · 精选路线 · 我的行程 · 打卡 · 内容后台 · 云开发

版本	**0.8.8（体验版）**
技术栈	**微信原生小程序 / Node 本地后台 / 微信云开发 / Python 数据管线**
数据规模	**51 个点位 • 7 条路线 • 283 对实测交通 • 45 张实景图**
代码仓库	**github.com/danta2006/map**
报告日期	**2026 年 10 月 29 日**

本报告覆盖：系统架构、前端 5 个页面、后端（本地内容后台 + 微信云开发）、数据生产管线、质量保障与已知限制

项	内容
项目性质	大学生创新创业训练项目 / 课程作品（非商业，演示为主）
代码仓库	https://github.com/danta2006/map（公开）
技术栈	微信原生小程序（WXML/WXSS/JS）+ Node 本地内容后台 + 微信云开发 + Python 数据管线
当前版本	**0.8.8**（体验版）
数据规模	51 个点位 / 7 条路线 / 283 对实测交通 / 283 条真实路径 / 45 张实景图
主包体积	1.29 MB（上限 2 MB）
代码规模	小程序 41 文件 ~8.0 千行；后台 11 文件 ~4.5 千行；脚本 15 个 ~2.2 千行
云开发环境	`cloud1-d5ghzdcqba4c291f8`（免费试用）
报告日期	2026-10-29

1. 项目概述

1.1 目标

做一个**能在一张地图上把长株潭值得去的点标出来**的小程序：点开有攻略、能按精选路线逛、到了能打卡，还能自己拼一条「我的行程」。

不做第二个大众点评，只保证三件事：**点是真的、路线是能走的、演示是稳的。**

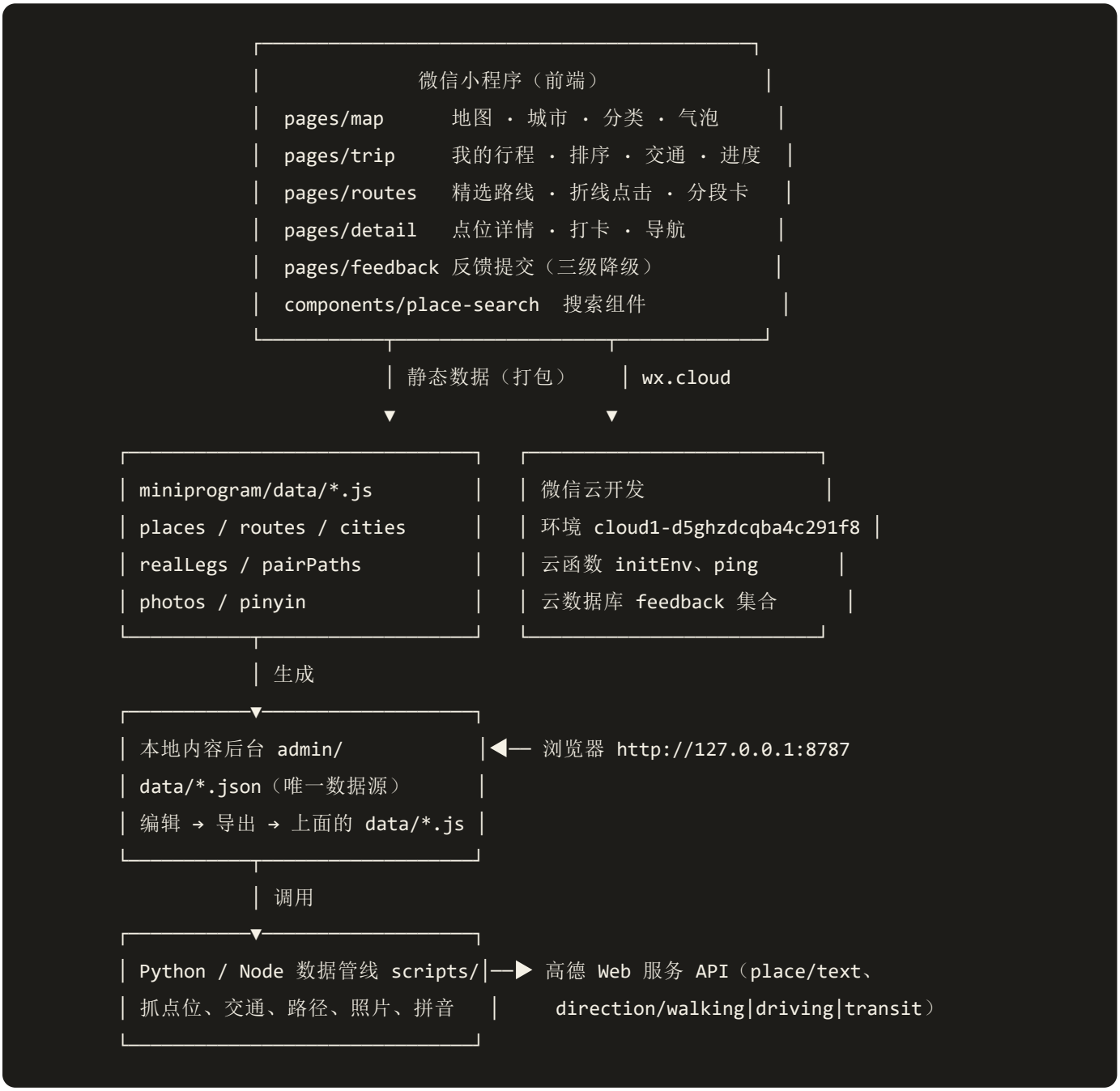
1.2 最终成果

能力	说明
三城地图	湘潭 26 / 长沙 16 / 株洲 9，共 51 个点位，6 个分类着色
点位信息	一句话、简介、本地备注、地址、开放时间、建议时长、实景图
精选路线	7 条（湘潭 3 / 长沙 2 / 株洲 2），沿道路真实折线 + 每段真实交通
我的行程	自选点位、拖拽排序、三种交通方式随时切换、打卡进度可视化
搜索	名称 / 标签 / 城市 / 分类 / 拼音全拼与首字母

能力	说明
打卡	按分类半径判定（300–1500 米），本地记录，可手动打卡兜底
内容后台	本地 Web 后台：点位/路线/城市可视化编辑、增删、一键重算、导出、反馈管理
用户反馈	详情页反馈入口 → 云端数据库（真机可用），三级降级保底
数据管线	高德接口批量拉点位 / 交通 / 路径 / 照片，脚本可重复执行

---

2. 系统架构



设计取舍：小程序端读**打包进去的静态数据**（而不是运行时请求后端），原因有三——

1. 个人主体无法备案 HTTPS 域名，真机请求外部接口会被拦；
2. 静态数据首屏无网络依赖，演示最稳；
3. 内容生产用本地后台，改完导出即可，不需要运维服务器。

云开发只承担「真机收集反馈」这一件事（云数据库不需要域名备案）。

---

3. 前端开发

3.1 工程结构

```
miniprogram/
├── app.js / app.json / app.wxss   全局（云开发初始化、导航、主题变量）
├── config.js                      环境配置（CLOUD_ENV / ADMIN_API）
├── data/                          数据层（6 个模块，约 90 KB）
├── utils/                         geo / travel / trip / search / api
├── components/place-search/       搜索组件
├── pages/
│   ├── map/      671 + 207 + 663 行    （js/wxml/wxss）
│   ├── trip/     697 + 214 + 747 行
│   ├── routes/   323 + 136 + 482 行
│   ├── detail/   219 + 66 + 247 行
│   └── feedback/  82 + 37 + 80 行
└── assets/                        分类 marker、时间图标、路线序号、实景图
```

3.2 地图首页（`pages/map`）

职责：总入口。看全城点位、切城市、筛分类、看点位卡片、进入详情或行程。

功能	实现方式
点位标记	`map` 的 `markers`，按分类取不同 pin 图，`callout` 显示名称（点击显示）
分类筛选	6 个 chip（全部/景点/古镇街区/红色/学府/美食/酒店），带分类色点当图例
城市切换	左上角城市 pill，下拉面板，切城市同时换 center/scale 与点位
点位卡片	底部卡片（名称/分类/时长/一句话/备注/导航），点卡片进详情

功能	实现方式
上滑详情面板	卡片可上滑展开整页详情（简介/地址/开放时间/距你/打卡），也可下滑收起
实景图气泡	缩放 ≥13.5 且与相邻点位屏幕间距 ≥128px 时，点位上方浮出圆形实景图气泡
加入行程	列表卡片右上「+」、面板标题「+ 行程」
搜索	右上 🔍 打开浮层，边输边出结果，结果先出介绍卡片再决定加入
定位	右下 ◎ `wx.getLocation` + `mapCtx.moveToLocation`

关键技术点

1. **气泡（customCallout）**：用 `map` 的 `<cover-view slot="callout">` + `marker-id` 绑定；显隐用 `marker.customCallout.display`。
   - 气泡尺寸 96rpx 圆形，`anchorX/anchorY` 控制位置（文档语义：**向右/向下为正**）；
   - 当前取值 `anchorX:6 / anchorY:10`，即正对图钉上方且底边贴住钉帽；
   - **坑**：`customCallout` 改变后地图不会自动重绘，必须把 `markers` 清空再写回强制重建；
   - 气泡本身也绑了 `bindtap`，点气泡等于点图钉。
2. **点击命中**：`bindmarkertap`（点图钉）、`bindcallouttap`（点原生气泡）、`bindtap`（空白处关闭），三者用 200–350ms 时间窗互斥，避免「卡片刚弹出就被关」。
3. **抖动治理**：地图中心/缩放**不做 regionchange 回写**（早期回写会导致拖动后地图自己跳），只在切城市/切分类/定位时主动设置；气泡重算用「显示集合 key 去重」，集合没变不 `setData`。
4. **城市切换**：`CITIES` 配置三城中心与缩放，切城市同时过滤点位、重置分类、收起卡片。

3.3 我的行程（`pages/trip`）

职责：用户自选点位拼行程，看总耗时/里程、调顺序、切交通方式、看打卡进度。

数据模型

```
// utils/trip.js，存 wx.storage: { ids: [...], modes: { 'a|b': 'walk'|'transit'|'taxi' }, updatedA
```

能力	实现
加点 / 移点	地图页「+」、详情页头图「+ 加入行程」、行程页搜索框；同一点位再点即移出

能力	实现
排序	**长按 260ms 进入拖动**，整个「站点块（站点 + 该段交通）」跟手移动，松手按最近块落位；也可用 ▲▼
地图绘制	编号 marker（1..N）+ 每段按交通方式着色的双层折线（白描边 13 + 主色 8）
交通耗时	每段列步行 / 公交地铁 / 打车三种时间，点选切换；有实测标「实测」
时间与里程	顶部统计：点位 / 游玩 / 在途 / 里程；里程用各段真实距离累加
打卡进度	「已打卡 N/M」+ 进度条 + 每个已打卡站点 ✓（读 `checkin_<id>` 记录）
面板	默认收起（只留底部一条），点开半屏（42% 屏高），可拖到全屏/隐藏
点路线弹卡	点折线**附近**、点两站之间的时间图标都能弹该段卡片，三种方式直接切

交通时间的三级数据源（`utils/travel.js`）

1. 精选路线里出现过的点对 → 直接用 `routes.js` 里高德返回的实测（`realSource: 'route'`）；
2. 同城 ≤8km 的其他点对 → 预计算的 `data/realLegs.js`（283 对，覆盖率 93%）；
3. 其余 → 直线距离 × 道路系数 1.28 + 各方式速度折算，界面标注估算。

跨城处理：两点不同城时，步行禁用（显示「跨城不推荐」），公交按城际列车估算（`距离/1500 m/min + 30min`），打车按高速估算（`距离/700 m/min + 5min`）；折线不画，改在中点标「跨城 XX 公里」。

3.4 精选路线（`pages/routes`）

能力	实现
路线分组	7 条路线按城市过滤，切城市同时切默认路线
真实折线	`routes.js` 的 `path`（高德 steps polyline 抽稀），双层描边
站点/交通切换	底部「站点 / 交通」二选一：横向站点卡 / 纵向逐段交通列表
点站点	弹「上一段 / 下一段」卡片 + 看详情按钮
点路线	`bindpolylinetap`（3.1.0+）+ `bindtap` 坐标兜底（按缩放换算容差，60–400m）
里程	各段 `legs[].distance` 累加

3.5 点位详情（`pages/detail`）

结构：头图（实景图 + 渐变遮罩，无图退化分类色块）→ 事实信息（分类/地址/开放时间/建议时长/距你）→ 关于这里 → 备注或补充 → 反馈链接 → 底部「导航 / 到这里打卡」。

打卡规则（`data/places.js` 的 `CATEGORY_RADIUS`，单点可用 `radius` 覆盖）

分类	半径	理由
学府	1500 m	一座校园从南门到北门就一公里
红色	1200 m	韶山冲这类成片景区
古镇街区	800 m	成片区域不是一个点
景点	600 m	一栋楼、一座公园
美食 / 酒店	300 m	到店门口

状态机：范围内直接打卡；范围外弹窗可选手动打卡；无定位引导设置或手动；>30km 额外提示疑似模拟定位。记录：`{ at, distance, manual }`。

3.6 反馈页（`pages/feedback`）

- 入口：详情页底部「发现信息有误？点这里反馈 ›」
- 表单：问题描述（≥5 字）+ 联系方式（选填）
- 提交链路（三级降级）：**云数据库** → **本地后台** → **本地存储**，成功提示区分「已提交到云端 / 已提交 / 已记录在本地」

3.7 数据层（`miniprogram/data/`）

文件	内容	体积
`places.js`	51 个点位 + 6 分类 + 分类打卡半径	33 KB
`routes.js`	7 条路线（stops / legs / path）	17 KB
`cities.js`	三城中心与缩放	0.5 KB
`realLegs.js`	283 对点位的真实交通时间（步行/打车/公交）	19 KB
`pairPaths.js`	283 条两点真实路径（抽稀后平均 13 点）	86 KB
`photos.js`	45 个点位实景图（圆形 marker 96px + 详情大图 320px）	3.6 KB

文件	内容	体积
`pinyin.js`	51 个点位拼音（全拼/首字母/含括号全拼）	4.2 KB

点位字段：

```
{ id, name, city, category, emoji, tags, latitude, longitude,
  summary, intro, note, address, openTime, durationMin }
```

3.8 视觉与交互规范

- 主题色：米白底 `#fbf8f2`、卡片 `#fffdf8`、主色红 `#b0392e`、青 `#2c6b63`
- 分类色：景点青 / 古镇街区褐 / 红色红 / 学府蓝 / 美食橙 / 酒店紫
- 圆角：卡片 20–32rpx，按钮 999rpx（胶囊）
- 动效：底部面板 `transform + cubic-bezier`，拖动时关 transition；气泡显隐 0.2s
- 文案：疑问句标题、口语化备注、不用感叹号

3.9 包体与性能

项	数值
主包	1.29 MB / 2 MB
其中实景图	约 900 KB（45 张圆形 134KB + 大图 763KB）
数据文件	约 163 KB
小程序代码	约 200 KB

优化手段：

1. 实景图 PNG 调色板量化（96px 从 12KB → 3KB）；
2. 详情大图 320px JPG q68（平均 17KB）；
3. 路径 Douglas-Peucker 抽稀（容差 ~33m，最多 60 点/条）；
4. 气泡重算去重（显示集合不变不 setData）；
5. 地图 markers 只在城市/分类/气泡状态变化时重建。

---

4. 后端开发

4.1 本地内容后台（`admin/`，11 文件 4.5 千行）

定位：内容生产后台（不是运行时接口）。零依赖 Node HTTP 服务，数据源是 JSON，导出成小程序静态数据。

```
admin/
├── server.js        HTTP 服务（API + 静态页面 + 重算脚本 + 云同步）
├── export.js        JSON → miniprogram/data/{places,routes,cities}.js
├── init-data.js     一次性：把现有 data/*.js 反向导成 JSON
├── data/*.json      唯一数据源（places/routes/cities/categories/feedback）
└── public/          原生 HTML/CSS/JS 管理界面
```

启动：`node admin/server.js` → http://127.0.0.1:8787

API 清单

方法	路径	作用
GET	`/api/data`	一次拿全部数据
POST	`/api/save`	`{name, data}` 写入 `admin/data/<name>.json`（含 feedback 状态修改）
POST	`/api/export`	生成小程序数据文件，返回各文件条目数
POST	`/api/feedback`	小程序提交反馈（带 CORS）
GET	`/api/feedback`	反馈列表
POST	`/api/refresh`	一键重算：`legs` / `paths` / `photos` / `all`（spawn Python）
POST	`/api/sync-cloud`	用 AppSecret 调云开发 HTTP API 拉取云端反馈
GET	`/api/health`	健康检查（含是否有 AMAP_KEY）

界面能力

页签	能力
精选路线	名称/城市/颜色/简介/站点顺序；新增路线；删除路线
点位	名称/emoji/分类/一句话/标签/简介/备注/地址/开放时间/时长/经纬度；搜索；新增；删除

页签	能力
城市	默认中心经纬度、缩放
用户反馈	列表、标记已处理/还原、刷新、从云端同步

数据流

```
界面编辑 → admin/data/*.json（唯一源）
                ├─ 导出 → miniprogram/data/*.js → 开发者工具编译
                └─ 重算 → scripts/*.py → data/realLegs.js / pairPaths.js / photos
```

安全与约束

- 只监听 127.0.0.1，不对外网暴露；
- 无鉴权（本地工具），文档注明不要部署到公网；
- AppSecret 只通过环境变量传入，不落盘、不入库。

4.2 微信云开发

项	内容
环境	`cloud1-d5ghzdcqba4c291f8`（免费试用）
云函数	`initEnv`（创建 feedback 集合，幂等）、`ping`（连通性自测）
数据库	`feedback` 集合，字段：`text/contact/placeId/placeName/page/at/status/_openid`
小程序初始化	`app.js` 读 `config.CLOUD_ENV`，非空才 `wx.cloud.init({ env, traceUser: true })`
部署方式	`cli cloud functions deploy --env <id> --names initEnv ping -r --project <path>`

反馈写入链路（`utils/api.js`）

```
reportFeedback(payload)
  ├─ cloudReady() → wx.cloud.database().collection('feedback').add(...)
  ├─ 失败 → POST /api/feedback（本地后台）
  └─ 再失败 → wx.storage 本地队列 + 提示
```

踩坑记录

1. 函数首次创建处于 `Creating` 状态，立刻部署会报 `FailedOperation.UpdateFunctionCode`，需等 15–20s 重试；
2. 不加 `-r`（云端安装依赖）时运行时报 `Cannot find module 'wx-server-sdk'`；
3. `customCallout` 更新不重绘（前端问题，见 3.2）；
4. 开通初期控制台 `tcb/DescribeEnvs` 与 `tcbDescribeUserPromotionalActivity` 均报 `ret:1000 system error`，重启开发者工具 3 次后恢复。

4.3 数据生产管线（`scripts/`，15 个脚本 2.2 千行）

脚本	输入	输出	依赖
`fetch_pois.py`	高德 place/text（多关键词分组）	`docs/采集草稿-*.xlsx`	AMAP_KEY
`make_form.py`	草稿 + 现有点位	`docs/长株潭点位采集表-v3.xlsx`（168 行）	node
`import_places.py`	采集表 v3	`miniprogram/data/places.js`	node
`fetch_legs.py`	routes.js 的站点序列	`_tmp/legs.json`（每段步行/公交/打车）	AMAP_KEY
`fetch_route_paths.py`	同上	`route_paths.json` + `leg_paths.json`	AMAP_KEY
`fetch_real_legs.py`	同城 ≤8km 点对	`data/realLegs.js`（283 对）	AMAP_KEY
`clean_real_legs.js`	realLegs.js	清洗后（剔除 45 个不合理值）	node
`fetch_pair_paths.py`	realLegs 点对	`data/pairPaths.js`（283 条，86KB）	AMAP_KEY
`fetch_photos.py`	点位的 POI photos 字段	`assets/photos-src/<id>.jpg`（45 张）	AMAP_KEY
`make_photo_markers.py`	photos-src	圆形 marker + 详情大图 + `photos.js`	Pillow
`build_pinyin.py`	places.js	`data/pinyin.js`	pypinyin
`pull_cloud_feedback.js`	云开发 HTTP API	`admin/data/feedback.json`	AppID/AppSecret

脚本	输入	输出	依赖
`check_pages.js`	全部页面 wxml/js	事件绑定/标签闭合检查报告	node
`gen_hotel_marker.py` / `gen_trip_markers.py`	—	酒店 marker、路线序号 7–20	Pillow

外部接口调用量（本次开发累计）：place/text 约 1400 次（三城关键词分组分页）、direction 约 1200 次（路线 + 点对交通 + 点对路径）、photos 45 次。高德个人配额 5000 次/日，未超限。

---

5. 数据资产

内容	规模	来源
点位	51（湘潭 26 / 长沙 16 / 株洲 9）	高德地点搜索 + 人工筛选文案
分类	6（景点 25 / 古镇街区 6 / 红色 6 / 学府 4 / 美食 4 / 酒店 6）	人工定义
路线	7 条，共 26 站、19 段交通、245 个折线点	人工设计 + 高德路径
两点实测	283 对（步行 134 / 打车 263 / 公交 205，三方式齐全 59）	高德路线规划
两点路径	283 条（平均 13 点/条）	高德路线规划
实景图	45 张（6 家酒店暂缺）	高德 POI photos
拼音索引	51 条	pypinyin
采集表	v3 共 168 行（湘潭 65 / 长沙 55 / 株洲 48）	脚本从 1381 条草稿筛出

---

6. 质量保障

6.1 静态检查（`scripts/check_pages.js`）

对每个页面检查：

1. WXML 里绑定的事件（`bindtap` / `catchtap` / `bindtouchmove` 等）在 JS 中是否有对应函数；
2. 标签闭合（含 `<image></image>` 等自闭合标签的特殊处理）。

> 这个脚本是因为一次误删 `toggleCityPicker` 导致「点城市按钮没反应」才加的，现在是上传前必跑项。

6.2 自动化验证

用 `miniprogram-automator` 连接开发者工具，做了以下验证（均通过）：

用例	结果
三城切换（地图页/路线页）	湘潭 26 / 长沙 16 / 株洲 9 正确切换
分类筛选（含酒店）	酒店 2 个点
加入/移出行程	tripCount、行程页站点数同步
行程排序（长按整块拖动）	顺序正确变更，编号图钉同步
交通方式切换	行程页折线颜色、统计同步
打卡进度	1/3 → 33% 进度条正确
点路线弹卡	折线附近 / 时间图标 / 精确折线三种方式均弹卡
气泡显隐	放大 18 个气泡 → 缩回 0 个
反馈提交	开发者工具 → 后台入库；云开发 → 云数据库入库
采集表导入	湘潭 24 行导入，其它城市不丢

6.3 已知问题闭环

问题	状态
27 个点位坐标错误（湘大偏 6.2km）	已用高德接口重取
城市切换点了没反应	误删事件函数，已修复并加静态检查
卡片互相堆叠	统一 `setCard()`，两类卡片互斥
气泡不显示	`regionchange` 的 `e.scale` 不可靠，改 `mapCtx.getScale()`
气泡收不起来	`customCallout` 不重绘，改为清空 markers 强制重建
标点变图片不可读	回退彩色 pin，图片只放气泡

问题	状态
行程画直线	补 283 条真实路径，跨城不画线
跨城出现「步行 631 分钟」	跨城步行禁用，改城际估算
云开发 `ret:1000`	微信侧抖动，重启后恢复
云函数缺依赖	部署加 `-r` 云端安装依赖

7. 已知限制

1. **没有运行时后端**：小程序读静态数据，改内容要重新导出、重新上传体验版（这是为绕开备案域名做的取舍）。
2. **个人主体限制**：不能选「旅游-景区服务」类目，不能发布上线；`wx.getLocation` 正式接口权限未申请，真机依赖授权弹窗与手动打卡兜底。
3. **图片版权**：45 张实景图来自高德 POI，**仅限课程演示**，商用/公开发布需替换为自有素材（替换流程已脚本化）。
4. **交通时间为静态预计算**：不含实时路况，早晚高峰驾车会偏乐观；跨城时间为估算。
5. **云开发为免费试用**：到期需续费（¥19.9/月）或迁回本地，数据在环境内。
6. **6 家酒店暂无实景图**（气泡退化为分类 pin）。
7. **我的页 / 足迹页未做**：打卡记录只在本机，未做汇总页。
8. **后台无鉴权**：仅限本机使用，不能部署公网。

8. 后续计划

优先级	事项	预计
P0	真机回归测试（三城、行程、打卡、反馈）	0.5 天
P0	答辩材料：演示脚本、PPT、架构图	1 天
P1	数据补量：长沙/株洲各补到 30+ 点（团队回填采集表）	2–3 天
P1	6 家酒店补图；图片素材逐步替换为自有拍摄	1 天
P2	「我的」页：足迹地图、按城市/分类完成度	1 天

优先级	事项	预计
P2	路线页支持自定义路线（从行程一键生成精选路线）	1 天
P3	云开发续费或迁移方案确定	到期前
P3	后台加登录口令 + 局域网访问（便于多人协作）	1 天

---

9. 附录

9.1 文件结构

```
微信小程序/
├── project.config.json      开发者工具配置（appid、cloudfunctionRoot）
├── miniprogram/             小程序源码（41 文件 1293 KB）
├── cloudfunctions/          云函数 initEnv / ping
├── admin/                   本地内容后台（11 文件 122 KB）
├── scripts/                 数据管线（15 个脚本）
├── docs/                    采集表、方案、截图、本报告
└── assets/photos-src/       照片原图（不打包、gitignore）
```

9.2 命令速查

```
# 小程序编译 / 上传
cli open --project "C:\微信小程序"
cli upload --project "C:\微信小程序" -v 0.8.9 -d "说明"

# 内容后台
node admin/server.js                # http://127.0.0.1:8787
node admin/export.js                # JSON → 小程序数据

# 数据管线（需要 $env:AMAP_KEY）
python scripts/fetch_real_legs.py
python scripts/fetch_pair_paths.py
python scripts/fetch_photos.py
python scripts/make_photo_markers.py
python scripts/build_pinyin.py

# 检查 & 部署云函数
node scripts/check_pages.js
cli cloud functions deploy --env cloud1-d5ghzdcqba4c291f8 --names initEnv ping -r --project "C:\微
```

9.3 版本记录

版本	内容
0.1.0–0.1.1	骨架：地图 + 21 点 + 筛选 + 卡片 + 详情 + 打卡降级
0.2.x	卡片交互重做、打卡半径按分类、路线页 3 条
0.3.0–0.3.2	城市切换；坐标校准（湘大偏 6.2km）；补 18 个知名点至 45 个
0.4.0–0.4.9	路线按城市拆分、真实交通、真实折线、路段点击与命中优化
0.5.0–0.5.11	卡片互斥、上滑详情面板、分类色点、实景图 marker→气泡、气泡点击
0.6.0–0.8.0	合入合作方版本：我的行程、点位搜索、酒店分类（51 点）
0.8.1–0.8.6	真实交通预计算、拼音搜索、打卡进度、行程真实路径、分段卡片、跨城修正
0.8.7	本地内容后台扩展：增删点位/路线、一键重算、用户反馈
0.8.8	微信云开发接通：反馈写云数据库（真机可用）

9.4 术语

术语	含义
GCJ-02	国测局坐标系，微信小程序地图与高德接口同一套
customCallout	地图 marker 的自定义气泡槽（cover-view）
legs	路线中相邻两站的交通段（方式/距离/时间/路径）
pairPaths	任意两点之间的真实路径（用于行程折线）
实测 / 估算	实测=高德路线规划返回；估算=距离×系数折算